Testiranje softvera

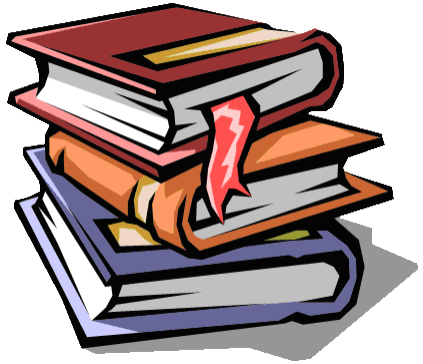


Autor: Goran Savić

Osnovni pojmovi



Literatura



- Knjiga
  - Spillner, A., Linz, T., Schaefer, H., **Software Testing Foundations**, 4th Edition, Rocky Nook, 2014
  - Savić, G., Segedinac, M., **Tehnologije veb aplikacija**, FTN izdavaštvo, 2018



Bog ih postavi u prostor nebeski da obasjavaju zemlju i da upravljaju danom i noću i da dele svetlost od tame. ***I Bog vide da je to dobro.***

(Post. 1, 17-18)

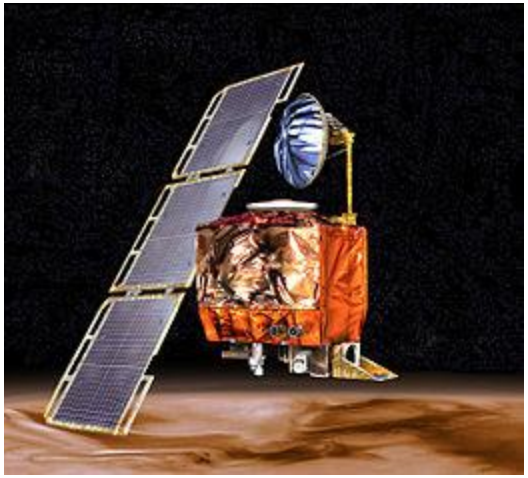
Primeri softverskih grešaka

- Lažna uzbuna o nuklearnom napadu (1983.)
  - ruski softver za detekciju nuklearnih napada greškom je prijavio nuklearni napad iz Amerike
  - planom je bio predviđen momentalni kontraudar
  - pukovnik Stanislav Petrov je primio upozorenje i ignorisao ga više verujući svom iskustvu nego softveru



Primeri softverskih grešaka

- Marsov satelit (1998.)
  - srušio se pri prilasku Marsu
  - šteta 327.6 miliona dolara
  - bag je bio to što nisu korišćenje iste jedinice mere u svim delovima softvera



Primeri softverskih grešaka

- Software horror stories
  - https://www.cs.tau.ac.il/~nachumd/horror.html
- Epic failures: 11 infamous software bugs
  - https://www.computerworld.com/article/1524595/epic-failures-11-infamous-software-bugs.html

Pojam testiranja



- **Testiranje**
  - Proces koji se sastoji od svih aktivnosti u životnom ciklusu softvera vezanih za planiranje, pripremu i izvršavanje zadataka koji treba da pokažu da li softver zadovoljava specificirane zahteve i ispunjava namenu, kao i da detektuju greške u softveru

Osnovni pojmovi

- Pri razvoju softvera neophodno je utvrditi da li softver zadovoljava postavljene zahteve
  - kvalitet softvera je određen stepenom usklađenosti sa zahtevima i korisnikovim očekivanjima
- Sva odstupanja od željenog kvaliteta moraju se ispraviti promenom
  - samog procesa razvoja
  - softverskog proizvoda

Osnovni pojmovi

- Procena kvaliteta softvera vrši se
  - statički
    - analizom procesa razvoja, dizajna i programskog koda
  - dinamički
    - izvršavanjem softvera kroz praćenje ponašanja i podataka u toku izvršavanja
- Testiranje softvera treba da **smanji rizik** od neželjenog funkcionisanja softvera
  - time što testiranje može da otkrije anomalije u softveru

Otkaz

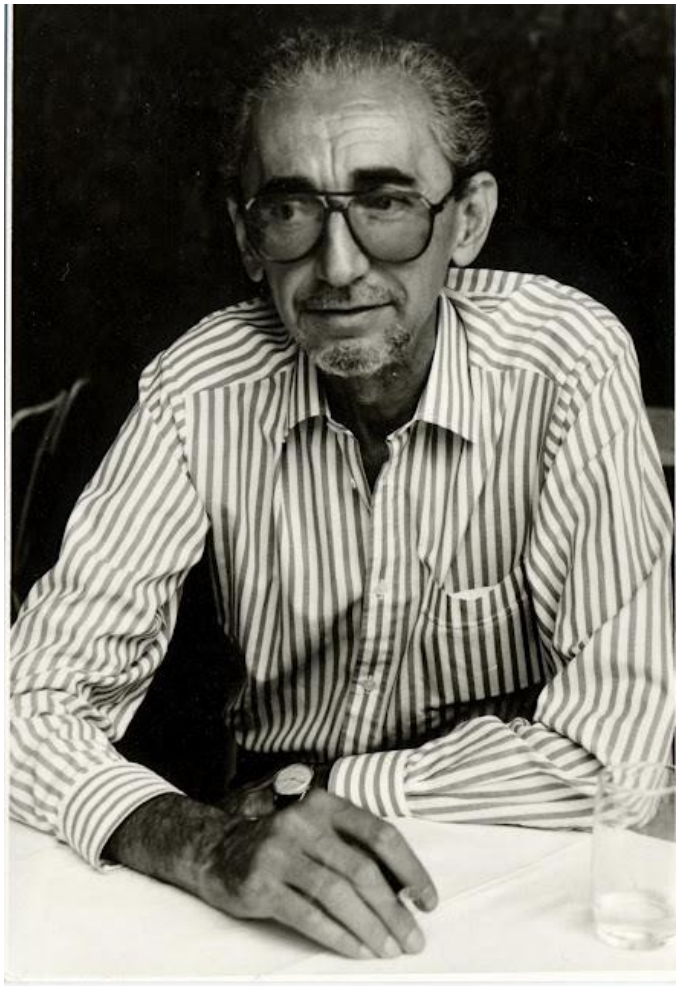
- Da bismo utvrdili šta je neočekivano ponašanje moramo znati šta je očekivano

- Testiranje se svodi na poređenje očekivanog i stvarnog ponašanja pri pozivu određenih funkcionalnosti koristeći **određeni uzorak** mogućih ulaznih podataka
  - nikad ne možemo proveriti sve moguće kombinacije podataka i funkcionalnosti



- **Otkaz (*failure*)**
  - Odstupanje softverske komponente od njenog očekivanog funkcionisanja
    - ne samo u isporučenom rezultatu, nego i u brzini isporučivanja rezultata, lakoći korišćenja, ...
  - Otkaz je posledica nedostatka u programu

Uzrok otkaza



Psihijatar zna sve, ali ne čini ništa. Hirurg ne zna ništa, ali čini sve. Dermatolog također ništa ne zna, ali ne čini ništa, jedino patolog sve zna, samo - jedan dan prekasno.
(Besnilo)

Borislav Pekić
(1930 – 1992)

Nedostatak



- **Nedostatak (*fault, defect, bug*)**
  - mana u programu koja može da onemogući program da izvrši namenjenu funkcionalnost
    - ukoliko se pri izvršavanju programa izvrši ovaj deo koda
- Nedostatak se manifestuje pojavom otkaza
  - u toku testiranja ili
  - korišćenja

Prikriveni nedostaci

- Nedostatak može biti prikriven drugim nedostacima
  - nedostatak ne može da se manifestuje zbog postojanja drugih nedostataka
  - manifestovaće se nakon uklanjanja nedostataka koji ga prikrivaju
    - to znači da se uklanjanjem nedostataka mogu pojaviti novi nedostaci kojih do tada nismo bili svesni

Greške u programu

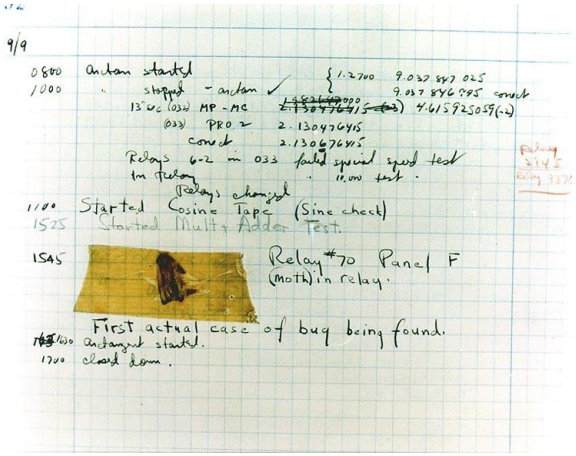
- Uzrok nedostatka u programu je najčešće
  - greška unešena u program od strane programera

- Zašto se greške dešavaju?
  - zbog nepažnje (posebno pri nedostatku vremena)
  - zbog nepoznavanja funkcionisanja programskog koda koji se unosi
  - zbog nepoznavanja posledica koje unešeni programski kod ima na ostatak programa
  - zbog kasnijeg funkcionisanja programa u okruženju čije se karakteristike nisu dobro predvidele ili se o njima nije razmišljalo
  - ...

Poreklo termina *bug*



- 1945. u računarskoj laboratoriji na Harvardu utvrđeno je da računar neispravno radi zbog moljca koji se uvukao u računar
  - moljac je uklonjen i ubačen u dokumentaciju projekta



- Odatle i termin *debugging* za proces otkrivanja bagova

Otkrivanje nedostataka



Popravio sam bugove, sada su mnogo bolji.

- Da bi se otkaz rešio, potrebno je
  - locirati nedostatak koji uzrokuje otkaz
  - ispraviti nedostatak
- Ispravljanje nedostatka u principu povećava kvalitet softvera
  - izmena najčešće ne uvodi nove nedostatke
- Ispravljanje nedostatka može da ima bočne efekte
  - pojavljuju se novi otkazi
  - iz tog razloga nakon izmene je važno testirati ne samo deo aplikacije koji je imao otkaz, nego i ostale delove

4 plemenite istine

Patnja postoji

Žudnja dovodi do patnje

Otklanjanjem žudnje nestaje patnja

Metod nestanka patnje je plemeniti osmostruki put



Buda (566 p.n.e – 486 p.n.e)

Nedostaci postoje

Greška dovodi do nedostatka

Otklanjanjem greške nestaje nedostatak

Metod nestanka nedostatka je plemeniti put uključivanja testiranja u proces razvoja softvera

Test proces



- **Test proces** obuhvata sve aktivnosti vezane za testiranje neke softverske komponente. Ovo uključuje planiranje, dizajn, implementaciju i izvršavanje testa, kao i evaluaciju rezultata i izveštavanje o rezultatima
  - pored navedenih, uključuje i druge aktivnosti, npr. procenu da li je potrebno dalje testiranje ili se testiranje te funkcionalnosti može završiti (opet samo privremeno do izmena u programu)

Test objekat



- **Test objekat**
  - softverska komponenta koja se testira

- Testiranje je svako izvršavanje ili analiziranje test objekta kako bi se utvrdilo njegovo ponašanje
  - porede se stvarno i očekivano ponašanje da bi se utvrdilo da li test objekat zadovoljava zahteve



- **Uslovi testa (*test conditions*)**
  - koje osobine ili funkcionalnosti test objekta treba verifikovati
  - verifikacija se vrši kroz test slučajeve

Test slučaj

- Konkretan postupak koji vrši verifikaciju određenog uslova testa



- **Test slučaj** čini skup ulaznih vrednosti, preduslova za izvršavanje, očekivanih rezultata i posledica izvršavanja određene softverske komponente koji treba da verifikuju njenu funkcionalnost

A QA Tester walks into a bar:

He orders a beer.

He orders 3 beers.

He orders 2976412836 beers.

He orders 0 beers.

He orders -1 beer.

He orders q beers.

He orders nothing.

Él ordena una cerveza.

He orders a deer.

He tries to leave without paying.

He starts ordering a beer, then throws himself through the window half way through.

He orders a beer, gets his receipt, then tries to go back.

He orders a beer, goes to the door of the bar, throws a handful of cookies into the street, then goes back to the bar to see if the barmaid still recognises him.

He orders a beer, and watches very carefully while the barmaid puts his order into the till to make sure nothing in his request got lost along the way.

He starts ordering a beer, and tries to talk the barmaid into handing over her personal details.

He orders a beer, sneaks into the back, turns off the power to the till, and waits to see how the barmaid reacts, and what she says to him.

He orders a beer while calling in thousands of robots to order a beer at exactly the same time.

Glavni elementi test slučaja

- Funkcionalnost koja se testira (*test condition*)
  - npr. prijavljivanje korisnika
- Preduslovi za izvršavanje
  - npr. da postoji u bazi podataka evidentiran korisnik sa korisničkim imenom *petar* i lozinkom *12345*
- Ulazni podaci
  - podaci koji se šalju delu aplikacije koji se testira
  - npr. korisničko ime *petar* i lozinka *12345*
- Očekivani rezultat i posledice
  - koje podatke/ponašanje očekujemo nakon izvršavanja test slučaja
  - Npr. korisnik je uspešno ulogovan

Evidencija test slučaja

- Test slučaj se može evidentirati kao dokument
  - sadrži podatke koji opisuju elemente test slučaja
  - za automatske testove koji se obavljaju programski, često je programski kod jedini „dokument“
- Treba dokumentovati i svako izvršavanje test slučaja
  - sadrži podatke o datumu izvršavanja, rezultatu izvršavanja, korisniku koji je izvršio test slučaj, ...
  - moguće je ove podatke i programski generisati ako se test slučaj izvršava automatski (programski)



- Primer
  - Primeri/test_case_report
  - Primeri/test_case_report_example

Test scenario

- Termin **test scenario** se koristi sa različitim značenjima
  - kao apstraktan opis test slučaja
  - kao hipotetička priča koja opisuje kako funkcioniše određeni tok u aplikaciji

Test scenario kao apstrakcija test slučaja



- **Test scenario** je opis ponašanja jedne funkcionalnosti
  - može se posmatrati kao test slučaj na višem nivou apstrakcije
  - ne bavi se konkretnim podacima kojima ćemo verifikovati funkcionalnost
  - u ovom značenju je blizak značenju test uslova
- Primer test scenarija
  - Kada korisnik unese neispravno korisničko ime, zabranjuje mu se prelazak na sledeći korak registracije

Test slučajevi na bazi test scenarija

- Na osnovu prethodnog test scenarija izrađuje se određeni broj test slučajeva koji verifikuju taj scenario
- Primer test slučajeva na osnovu test scenarija
  1. Provera dužine korisničkog imena
     - unesemo u tekstualno polje vrednost `ab`
     - očekivani rezultat je zabrana registracije jer je korisničko ime prekratko
  2. Provera sadržaja korisničkog imena
     - unesemo u tekstualno polje vrednost `abcdefgh`
     - očekivani rezultat je zabrana registracije jer korisničko ime ne sadrži cifre

Test scenario kao tok u aplikaciji



- **Test scenario** je hipotetička priča koja kroz tok aktivnosti opisuje kako funkcioniše određeni deo aplikacije
- Primer test scenarija
  1. korisnik vrši logovanje u aplikaciji
  2. nakon uspešnog logovanja otvara pregled svojih uplata
  3. na stranici sa spiskom uplata unosi novu uplatu
  4. nova uplata se pojavljuje u spisku svih uplata
- Na osnovu ovog scenarija izrađuje se skup test slučajeva, koji kad se izvrše u nizu verifikuju scenario

Test suite



- **Test suite** predstavlja agregaciju test slučajeva
  - Test slučajevi se grupišu da bi se izvršavali kao jedna celina
  - Grupišu se logički bliski test slučajevi, ali ne moraju da čine tok korisničkih aktivnosti kao kod test slučajeva koji čine test scenario

Kvalitet softvera



Kvalitet softvera

- Testiranje se vrši kako bi se unapredio kvalitet softvera
  - unapređenje se vrši identifikacijom nedostataka i njihovom ispravkom
  - ako test slučajevi pokrivaju reprezentativan skup scenarija korišćenja softvera, onda će korisnik dobiti kvalitet softvera koji je utvrđen testiranjem

Faktori kvaliteta softvera

- Kvalitet softvera se može meriti sledećim faktorima (ISO 9126)
  - funkcionalnost
  - pouzdanost
  - upotrebljivost
  - efikasnost
  - mogućnost održavanja
  - prenosivost

- Testiranjem se ocenjuje kvalitet nekih od ovih karakteristika
  - svaki test je usmeren na ocenu neke od karakteristika
  - potrebno je za svaku karakteristiku odrediti koji nivo kvaliteta želimo da obezbedimo testiranjem

Funkcionalnost



- Funkcionalnost predstavlja sposobnost softvera da obavi funkciju za koju je namenjen
  - Definiše šta sistem treba da uradi
- Ostale karakteristike kvaliteta ima smisla analizirati samo ako je softver funkcionalan
- Sposobnost obavljanja funkcionalnosti se meri reakcijom softvera na zadate ulaze

Testiranje funkcionalnosti

- Testiranje funkcionalnosti treba da pokaže da li je određeni funkcionalni zahtev implementiran u skladu sa specifikacijom
  - najvažnije je proveriti ispravnost rada komponente

- U sklopu testiranja funkcionalnosti treba proveriti i
  - kako komponenta komunicira sa drugim komponentama/sistemima
  - usklađenost načina korišćenja sa zakonima i standardima
  - bezbednost aplikacije

Pouzdanost



- Pouzdanost predstavlja sposobnost softvera da zadrži traženi nivo funkcionisanja u određenim okolnostima korišćenja u toku određenog perioda

- Pouzdanost definišu
  - zrelost
    - koliko često se otkazi dešavaju kao rezultat grešaka u softveru
  - otpornost na greške
    - sposobnost softera da održava zahtevani nivo performanse ili da se oporavi od grešaka
      - koje se nalaze u softveru ili koje su uzrokovane okruženjem ili nepravilnim korišćenjem i podacima
  - sposobnost oporavka
    - sposobnost ponovnog uspostavljanja zahtevanog nivoa performansi i podataka, nakon što se otkaz desi
      - meri se vremenom i količinom posla potrebnim za oporavak, kao i kvalitetom usluge i podataka nakon oporavka

Upotrebljivost



- Upotrebljivost je sposobnost softvera da funkcionalnosti pruža na način pogodan korisniku
  - ovde nije naglasak samo šta softver radi, nego i kako radi
- Aspekti upotrebljivosti
  - razumljivost
  - lakoća korišćenja
  - lakoća savladavanja
  - atraktivnost
  - usklađenost sa standardima, konvencijama, stilovima i drugim aspektima korisničkog interfejsa
- Ova karakteristika se meri nefunkcionalnim testovima

Efikasnost



- Efikasnost je sposobnost softvera da funkcionalnosti pruža u skladu sa traženim performansama
  - brzina izvršavanja, vreme odziva, potrošnja memorije

- Test efikasnosti meri potrebno vreme i potrošnju resursa pri izvršavanju nekog zadatka
  - resursi uključuju softverske i hardverske komponente

Lakoća održavanja



- Lakoća održavanja se odnosi na to koliko lako softver može biti izmenjen da bi se ispravili nedostaci, ispunili novi zahtevi, softver prilagodio novom okruženju, kao i refaktorizovao kako bi buduće održavanje bilo jednostavnije
- Potkarakteristike ove karakteristike su
  - lakoća analize
    - čitljivost koda značajno utiče na ovu potkarakteristiku
    - korišćenje široko prihvaćenih šablona i radnih okvira povećava lakoću analize
  - lakoća izmene
  - stabilnost
    - odnosi se na verovatnoću da će biti potrebno uvoditi izmene
  - lakoća testiranja
    - nije svako rešenje jednako jednostavno za testiranje
- Uglavnom se ove karakteristike testiraju statičkim testiranjem (analiza softvera bez izvršavanja)

Prenosivost



- Prenosivost je sposobnost softvera da zadrži nivo kvaliteta pri pokretanju u drugačijem okruženju

- Potkarakteristike koje se odnose na prenosivost
  - prilagodljivost
  - lakoća instalacije
  - usaglašenost
    - sa standardima, konvencijama, široko prihvaćenim rešenjima
  - zamenljivost komponenti
    - na primer operativni sistemi podržavaju različite hardverske uređaje putem drajvera koji su lako zamenljivi

Intenzitet testiranja



Intenzitet testiranja

- Nesporno je da je testiranje aplikacije neophodno

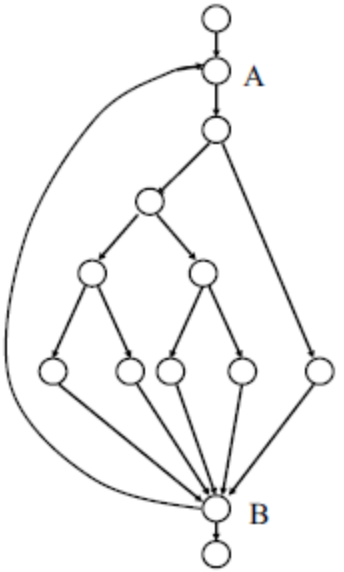
- Koliko testiranja je potrebno?
  - razvoj i izvršavanje testova troši resurse
    - ljudske, vremenske, softverske, hardverske, ...
  - premalo testiranja rezultuje nepouzdanom aplikacijom
  - previše testiranja može da ugrozi razvoj proizvoda ako ne postoje resursi za predviđenu količinu testiranja

Intenzitet testiranja

- Testiranje ne može da garantuje da greške u programu ne postoje
  - za ovakvu garanciju bi se morao izvršiti program u svakoj mogućoj situaciji sa svim mogućim ulaznim podacima u svim mogućim kombinacijama
  - zato testiranje samo može da pokaže da postoje greške
- Potpun test nije izvodiv
  - zbog broja mogućih kombinacija, broj testova bi bio praktično beskonačan

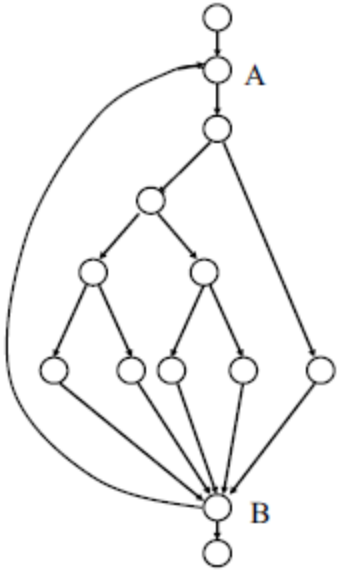
Primer potpunog testiranja

- Ako imamo jednostavan niz naredbi
  - neke su naredbe granjanja
  - između A i B je ciklus sa povratkom na A
- 5 mogućih putanja unutar jednog ciklusa
- Na kraju svakog ciklusa se može vratiti na A ako je uslov ispunjen



Primer potpunog testiranja

- Broj mogućih putanja izvršavanja ako je maksimalno moguće 20 iteracija
  - $5^{20} + 5^{19} + 5^{18} + \ldots + 5^{1}$
    - jer svaka putanja može da se vrati nazad na A
    - program može da ima 20 ili 19 ili 18 ili ...1 iteraciju
  - to je 119 209 289 550 780 mogućih putanja u programu



- Za manuelno testiranje sa 5 minuta po test slučaju bi trebala milijarda godina
- Za automatsko testiranje sa 5 μs po test slučaju bi trebalo 19 godina



Intenzitet testiranja

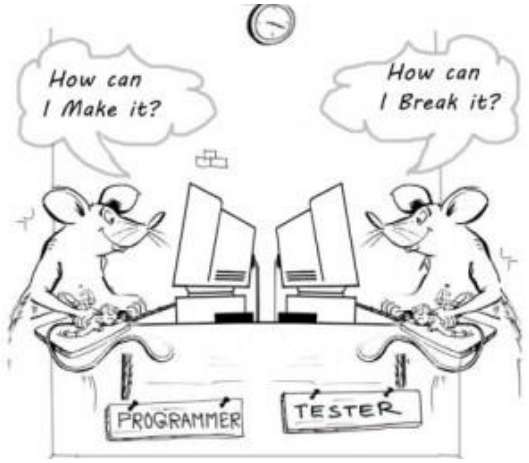
- Pošto je potpuno testiranje neizvodivo, testira se onoliko koliko je dovoljno
- Koliko je dovoljno?
  - meri se u odnosu na rizik otkaza u softveru
- Rizik se računa na osnovu verovatnoće pojave otkaza i očekivanog iznosa štete
- **Testiranje ne treba prekidati dok god je trošak nalaženja i ispravke nedostatka niži od troška koji proizvodi otkaz sistema zbog tog nedostatka**

Intenzitet testiranja



- Potrebno je naći balans u količini testova
  - mora postojati neophodan broj testova koji smanjuju verovatnoću otkaza
  - ne treba da postoje testovi koji ne utvrđuju nove informacije o kvalitetu softvera
  - voditi računa da ne dođe do eksplozije test slučajeva
    - time bi potrošili sve resurse na razvoj i izvršavanje testova
  - voditi računa o ograničenim resursima

Broj testera



- Odnos broja testera i programera zavisi od prirode projekta i upravljanja projektom
  - kreće se od 1 testera na 10 programera
  - u praksi je najčešće odnos 1 tester na 3 programera
  - ide i do više od 3 testera po programeru
- Nema generalnog pravila niti utvrđene prakse

> *To nije bug to je feature.*

Proces testiranja

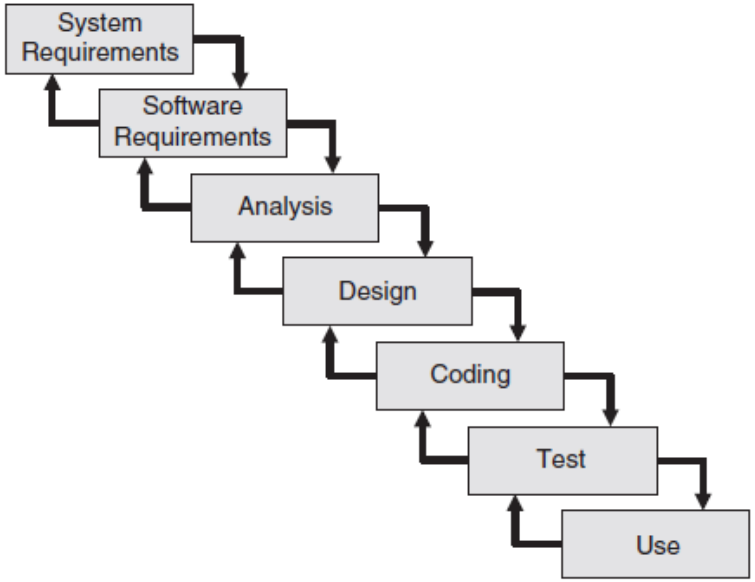


Mesto testiranja u životnom ciklusu razvoja softvera

- Testiranje ima svoje mesto u razvoju savremenih aplikacija
- Konkretno mesto zavisi od metodologije razvoja
- Svaka metodologija predviđa postojanje različitih faza u izradi softvera
- Testiranje se pojavljuje u svakoj od metodologija, ali u različitom značenju i sa različitom važnošću

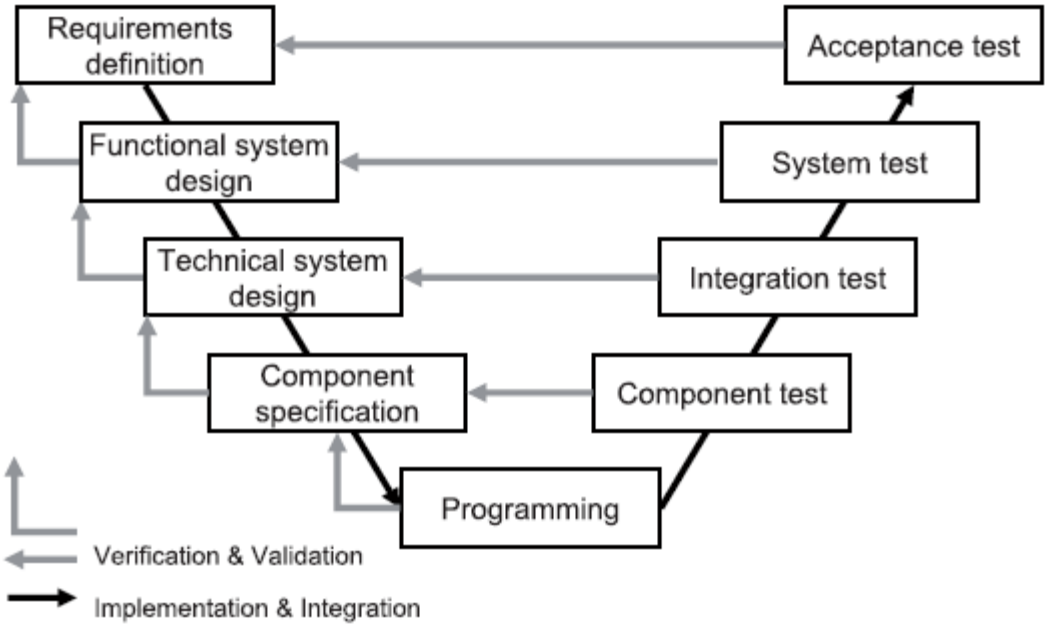
Metod vodopada

- Kod klasičnog „vodopad“ modela
  - predviđa se linearan prolazak kroz faze razvoja
  - testiranje se obavlja pred korišćenje proizvoda kao verifikacija implementirane aplikacije
  - mana ovog modela je što je testiranje izolovano u posebnu fazu koja se izvršava samo jednom i to na kraju



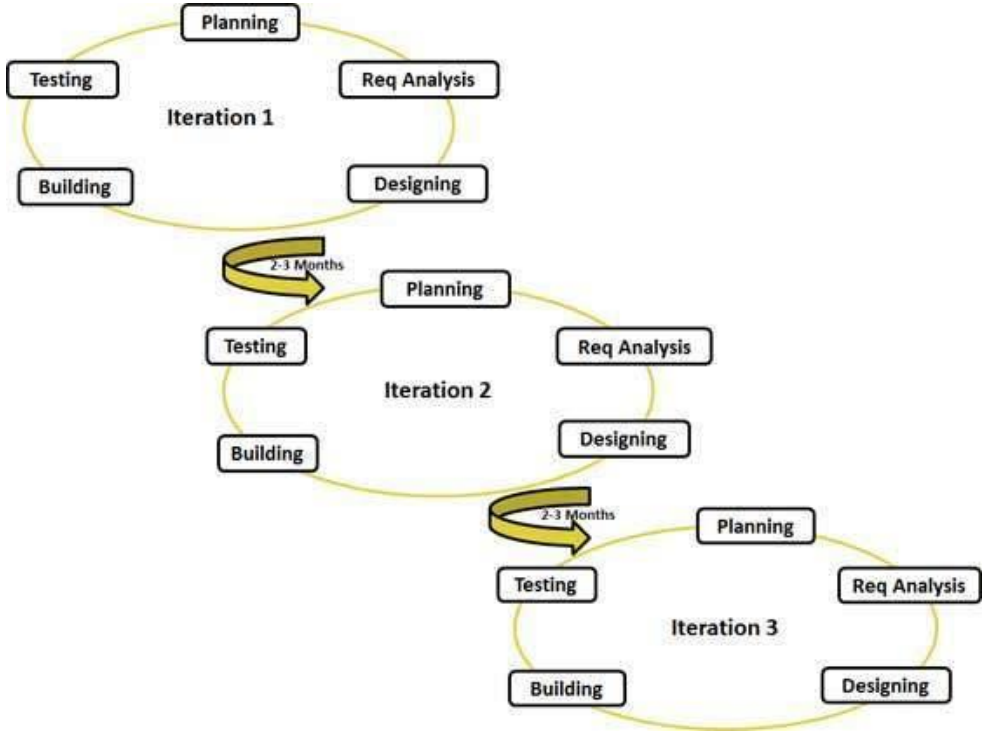
Opšti V-model

- Kod V-modela svaka faza razvoja ima odgovarajuću test fazu
  - na ovaj način posebnim tipom testova verifikuje se urađeno u svakoj od faza



Agilne metode razvoja softvera

- Predviđaju iterativno prolaženje kroz sve faze razvoja
- U svaku iteraciju uključeno je testiranje kao verifikacija urađenog



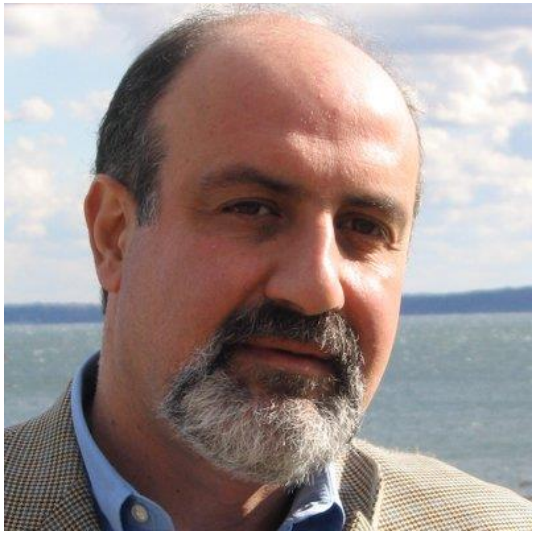
Razvoj softvera vođen testovima

- Klasično testiranje podrazumeva verifikaciju urađenog u nekoj od faza razvoja
- Razvoj softvera vođen testovima (*test-driven development* - TDD)
  - razvoj softvera kreće od pisanja testova
  - zahtevi se prevode u test slučajeve
  - nijedan test slučaj na početku ne prolazi uspešno jer sistem nije implementiran
  - implementacija treba da postigne da se svi test slučajevi uspešno izvrše

Razvoj softvera vođen testovima (TDD)

- TDD podstiče da se pri razvoju funkcionalnosti krene od njenog API-ja (zaglavlje metode ako je reč o testiranju metode)
  - Da bi se mogao napisati test, mora postojati API koji će iz testa biti pozvan
  - Nakon što je definisan API, tek onda se prelazi na njegovu implementaciju
  - Time programer prvo mora da precizira šta je ulaz i izlaz za tu funkcionalnost, pa tek onda da se fokusira na implementaciju algoritma koji transformiše ulaz u izlaz

TDD i antikrhkost



Nasim Nikolas Taleb

- Antikrhkost (eng. *antifragility)*
  - Osobina sistema da postaje jači ako je izložen stresu, šokovima, otkazima

TDD i antikrhkost



- Krhke stvari se lome ako su izložene stresu



- Otpornim stvarima stres ne smeta, ali im ni ne pomaže



- Antikrhkim stvarima stres pomaže tako što bivaju jače ako su izložene (umerenom) stresu

TDD i antikrhkost

- U pogledu uticaja grešaka na sistem
  - Krhak sistem „mrzi“ greške
  - Za otporan sistem greška je samo informacija
  - Antikrhak sistem „voli“ greške (ako su male), jer od njih postaje bolji

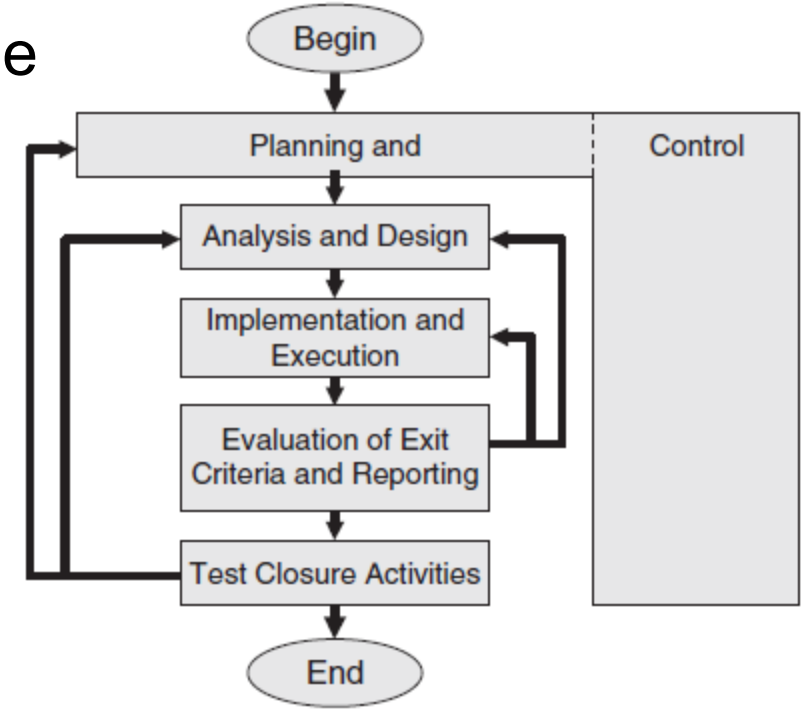
- Ako je sistem takav da se dešavaju nepopravljive, velike (ali retke) greške, sistem je krhak
- Ako je sistem takav da proizvodi popravljive i male greške, on je antikrhak
- U odsustvu čestih, sitnih grešaka, najverovatnije će retka greška kad nastupi biti velika i nepopravljiva

TDD i antikrhkost

- TDD pomaže da softver bude „antikrhak“
- TDD u razvoj softvera unosi puno sitnog, lokalizovanog i kontrolisanog nereda
  - Pre funkcionalnosti se prvo pišu testovi koji čine da softver ne prolazi verifikaciju dok se funkcionalnost ne implementira
  - Time se forsira da aplikacija ulazi kratkoročno u periode nestabilnosti kako bi dugoročno bila kvalitetnija

Koraci u testiranju

- Bez obzira na kojem mestu u razvoju softvera faza testiranja stoji, ona sadrži svoje podfaze
  - planiranje i upravljanje
  - analiza i dizajn
  - implementacija i izvršavanje
  - evaluacija i izveštavanje
  - završne aktivnosti



Faza planiranja

- Planiranje se vrši na više nivoa
- Na generalnom nivou



  - **Test strategija** definiše mesto i ciljeve testiranja u projektu, opšti pristup testiranju, nameravani intenzitet testiranja
  - Ista strategija se može primeniti na različite projekte
- Na specifičnijem nivou



  - **Test plan** opisuje raspored nameravanih aktivnosti testiranja kao i resurse koji će biti korišćeni u ovim aktivnostima

Upravljanje testiranjem

- Nadzor nad aktivnostima testiranja i kontrola kako se realizacija, dinamika i rezultat ovih aktivnosti uklapaju u plan testiranja
  - plan i aktivnosti testiranja se dinamički ažuriraju na osnovu prikupljenih informacija
- Upravljanje uključuje administraciju i održavanje
  - procesa testiranja
  - infrastrukure koja se koristi za testiranje (softverski alati, dokumenti, hardver)
- Ovakav nadzor mora biti uključen u svaki od koraka testiranja

Određivanje intenziteta testiranja

- U fazi planiranja se definiše planirani intenzitet testiranja
- Definisan je kriterijumom za završetak testiranja
- Kriterijum za završetak testiranja
  - skup uslova koji trebaju biti ispunjeni da bi se proces testiranja mogao smatrati završenim
  - svrha kriterijuma je da se testiranje završi tek kad određeni pokazatelji ukazuju da možemo smatrati aplikaciju pouzdanom



Kriterijumi za završetak testiranja

- Čest uslov koji se koristi kao kriterijum za završetak je stepen u kojem je softver pokriven testovima
- Pokrivenost testovima (*test coverage*)
  - Procenat delova koda za koje postoje testovi koji verifikuju njihov rad



- Mogu se koristiti i drugi kriterijumi
  - Npr. da svaki klijentov zahtev iz specifikacije bude testiran bar jednim testom

Utvrđivanje prioriteta testova

- Zbog ograničenih resursa, neophodno je klasifikovati testove po prioritetima
  - prioritetniji testovi se prvi realizuju
  - u nedostatku vremena za izvršavanje svih testova, izvršavaju se prioritetniji testovi

Alati za podršku

- Zbog efikasnijeg procesa testiranja potrebno je uključiti specijalizovane softverske alate
- U fazi planiranja potrebno je izabrati i pripremiti ove alate za upotrebu
  - može biti potrebna i dodatna obuka testera za korišćenje alata
- Postoje okruženja, tehnologije i alati za ovu svrhu
  - Junit, Selenium, Karma, Protractor
- Postoje i alati za podršku upravljanju testiranjem
  - alati za prijavu bugova, evidenciju zahteva, ...

Faza analize i dizajna

- Prvi korak u ovoj fazi je analiza baze testiranja



  - Baza testiranja
    - predstavljaju je svi dokumenti iz kojih se utvrđuju funkcionalnosti sistema
    - ove informacije su osnov za odluku šta treba biti testirano (uslovi testiranja)
- Zatim se kreira test specifikacija
  - sadrži test slučajeve koje treba izvršiti
  - test slučajevi se definišu na osnovu baze testiranja, strategije testiranja i prirode test objekta
  - važno je da postoji jasna veza između test slučaja i zahteva koji se testira tim test slučajem

Faza analize i dizajna

- Dizajn test slučajeva se vrši na dva nivoa
  - Logički test slučaj
    - definiše šta test slučaj testira i na generalnom nivou na koji način to treba da radi
  - Konkretan test slučaj
    - konkretne ulazne vrednosti se definišu
    - dizajn i implementacija konkretnih test slučajeva se vrši u sledećoj fazi
- Potrebno je grupisati test slučajeve u *test suites* i dizajnirati test scenarije

Faza analize i dizajna

- Test slučajevi mogu biti dizajnirani
  - na osnovu specifikacije funkcionalnosti test objekta
    - testiranje metodom crne kutije (*black box testing*)
  - na osnovu analize izvornog koda test objekta
    - testiranje metodom bele kutije (*white box testing*)

- Za svaki test slučaj se definišu
  - preduslovi koji moraju biti ispunjeni pre izvršavanja
  - očekivani rezultati i ponašanje

Faza analize i dizajna

- Potrebno je dizajnirati različite tipove test slučajeva
- Pozitivni testovi
  - šaljemo validne ulazne podatke i proveravamo da li sistem uspešno izvršava planiranu funkcionalnost
    - npr. pošaljemo validne kredencijale pri testiranju prijave na sistem
- Negativni testovi
  - šaljemo ulazne podatke na koje sistem treba da odgovori neizvršavanjem planirane funkcionalnosti
    - npr. pošaljemo kredencijale nepostojećeg korisnika pri testiranju prijave na sistem

Faza implementacije i izvršavanja

- U ovoj fazi se na osnovu logičkih test slučajeva implementiraju i izvršavaju konkretni test slučajevi, pri čemu se vrši evidencija o izvršavanju
- Manuelni testovi se odmah izvršavaju korišćenjem aplikacije
- Automatski testovi se implementiraju u definisanom programskom jeziku, najčešće korišćenjem specijalizovanih tehnologija
  - npr. testovi se pišu u programskom jeziku Java korišćenjem JUnit radnog okvira za testiranje

Faza implementacije i izvršavanja

- Prvi korak u izvršavanju testa je provera da li je test objekat spreman za testiranje
  - aplikacija se startuje i proveri da li su funkcionalnosti test objekta dostupne pri izvršavanju testova

Faza implementacije i izvršavanja



- Naredni korak je provera samo glavnih funkcionalnosti kroz *smoke test*
- ***Smoke test*** je jednostavan test koji samo proverava da li sistem „radi“
  - proverava da li je aplikacija startovana i da li su glavne funkcionalnosti prisutne bez ulaženja u detalje
  - ime potiče od osnovne provere ispravnosti električnih uređaja time da li će se zapaliti i pojaviti dim ako ih uključimo u struju
  - ako ovaj test ne prolazi nema smisla dalje izvršavati specifičnije testove



Svi električni uređaji rade na beli dim. Kada beli dim izađe, uređaj više ne radi.

Faza implementacije i izvršavanja

- Nakon *smoke* testa vrši se izvršavanje svih implementiranih testova
- Važno je evidentirati izvršavanje i rezultate testova (*test log*)
  - na osnovu toga se utvrđuju otkazi, pa pronalaze i ispravljaju nedostaci
- Svaki otkaz mora biti moguće reprodukovati ponavljanjem funkcionalnosti u istim uslovima
  - definisano testom, preduslovima i ulaznim podacima
  - programer nakon reprodukovanja otkaza utvrđuje da li nedostatak zaista postoji i otklanja ga
    - ponovnim testovima treba verifikovati ispravku i voditi računa da ispravka može uvesti nove otkaze

Faza ocenjivanja i izveštavanja

- U ovoj fazi se proverava da li su ispunjeni kriterijumi završetka testiranja i izveštava se o rezultatima testiranja
- Ako kriterijumi nisu ispunjeni, izvršavaju se dodatni test slučajevi ili se revidira kriterijum
  - potrebno je prepoznati da li novi test slučajevi doprinose verovatnoći pronalaženja nedostataka
- Kriterijum mora biti postavljen realno
  - može se meriti u odnosu na pokrivenost koda
  - ili na učestalost otkrivanja novih nedostataka pri testiranju
    - kada učestalost padne ispod određenog praga, može se testiranje prekinuti

Faza ocenjivanja i izveštavanja

- U plan razvoja softvera mora biti uračunato to da će proći više ciklusa testiranja dok ne bude ispunjen kriterijum završetka

- Nakon svakog testiranja formira se test izveštaj
  - sadrži podatke o izvršenim testovima
    - podaci o izvršenom test slučaju
    - uspešnost izvršavanja

Završne aktivnosti u testiranju

- U ovoj fazi vrši se analiza izvršenog procesa testiranja kako bi se prikupile informacije korisne za naredni ciklus testiranja ili za naredni projekat
  - treba uporediti planirane sa ostvarenim aktivnostima
    - koji ciljevi su odstupili od plana ili nisu uopšte realizovani
  - treba identifikovati probleme
    - koje neočivane situacije su se desile
  - treba izvesti zaključke šta treba izmeniti za buduća testiranja
- Potrebno je sačuvati test okruženje kako bi se
  - mogli izvršiti ponovni testovi
  - iskoristiti okruženje za buduće projekte

Psihologija testiranja



Psihologija testiranja

- Zadatak testera je da pronalazi nedostatke u programu
  - ovo se često pogrešno interpretira kao destruktivna aktivnost za razliku od konstruktivnog razvoja softvera

- Da bi se testiranje dobro uklopilo u životni ciklus softvera treba voditi računa o psihološkim faktorima u testiranju

Psihologija testiranja

- Ko bi trebao da testira određeni kod?

- Testiranje sopstvenog koda
  - obično nije najbolji izbor
  - programeri previde sopstvene greške i pretpostavke
    - ne razmišljaju o svim scenarijama korišćenja aplikacije
  - izbegavaju intenzivno testiranje jer svaki nedostatak donosi posao otklanjanja nedostatka



I'm always right. This time I'm just even more right than usual.

Linus Torvalds

Psihologija testiranja

- Nezavisni test tim
  - nezavisan tester pristupa drugačije softveru obzirom da nije njegov autor
    - nema psihološki problem da utvrdi da sistem ima nedostatke
  - nezavisni testeri su uspešniji u pronalaženju nedostataka
    - ima znanje testiranja pa su njegovi testovi kompletniji
    - razvijaju posebnu psihologiju korišćenja softvera tako da pronalaze scenarije koji mogu da izazovu otkaz
  - i kada postoji nezavisni test tim, često testove nižeg nivoa, koji se bave strukturom koda, pišu sami programeri jer je prezahtevno da se tester upoznaje sa kodom
    - zbog manjka vremene i resursa ili nepoznavanja tehnologije

Psihologija testiranja

- Za odnos programera i testera je važan način prijave nedostataka
  - posao testera je da ukazuje drugima da greše
  - programer ima lični odnos prema onom što realizuje
  - programer nekad lično shvata prijavu nedostatka



> *Ne nuntium necare (Nemoj ubiti glasnika)*
>
> Latinska izreka

- Da bi se smanjila mogućnost nesporazuma u komunikaciji
  - pre prijave nedostatka tester treba da bude siguran da je nedostatak u programu a ne u samom testu
  - mora da prijavi nedostatak na način da ga programer može reprodukovati kako bi zaista uvideo da nedostatak postoji

7 principa testiranja



Prvi princip

- ***Testiranje može da pokaže postojanje nedostataka, ali ne i odsustvo nedostataka***
  - ako test ne prolazi, to znači da nedostatak postoji
  - ako svi testovi prolaze, to i dalje ne dokazuje da nedostataka nema
  - testiranje samo smanjuje verovatnoću da nedostaci postoje

Drugi princip

- ***Potpuno testiranje je nemoguće***
  - nije moguće kreirati i izvršiti testove koji bi obuhvatili sve moguće ulazne vrednosti, i njihove kombinacije zajedno sa kombinacijama svih različitih preduslova
  - svaki test slučaj je samo uzorak iz skupa mogućih podataka i akcija
    - potrebno je pronaći podskup mogućih test slučajeva tako da u granicama dostupnih resursa obezbede što veću verovatnoću ispravnosti programa

Treći princip

- ***Aktivnosti vezane za testiranje treba otpočeti što ranije***
  - od samog starta dizajna softvera treba uračunati i testiranje i uključiti ga u sve faze razvoja softvera
  - što se nedostatak ranije pronađe, troškovi njegovog otklanjanja su manji

Četvrti princip

- ***Grupisanje nedostataka***
  - nedostaci nisu ravnomerno raspoređeni po programu
  - većina nedostataka se nalazi u pojedinim delovima test objekta
  - ako se u određenom delu programa pronađu nedostaci, najverovatnije ih ima još u tom delu programa

Peti princip

- ***Paradoks pesticida***
  - Kao što insekti postaju otporni na pesticide, tako više puta ponovljeno izvršavanje istih testova prestaje da doprinosi pronalaženju novih nedostataka
  - Potrebno je razvijati nove test slučajeve
    - novi nedostaci se verovatno nalaze u delovima koda i scenarijima koje postojeći test slučajevi ne pokrivaju

Šesti princip

- ***Testiranje zavisi od konteksta***
  - Testiranje mora biti prilagođeno nameni aplikacije, okolnostima u kojima se koristi i mogućem riziku koji otkaz izaziva
  - Različiti sistemi se ne testiraju na isti način
    - strategija testiranja, intenzitet testiranja, kriterijum završetka itd. moraju biti prilagođeni kontekstu konkretnog sistema

Sedmi princip

- ***Zabluda je misliti da je sistem bez nedostataka upotrebljiv***
  - Pronalaženje i otklanjanje nedostataka i dalje ne znači da sistem ispunjava korisnikova očekivanja
  - Korisnik ne očekuje samo ispravan sistem nego i sistem koji je vizuelno i funkcionalno za njega intuitivan
    - intuitivnost je dosta subjektivna kategorija, pa ju je teško ugraditi u specifikaciju zahteva
    - treba uvesti korisnika u rane faze razvoja aplikacije i izradom ranih prototipa dobiti korisne povratne informacije u pogledu njegovih očekivanja